# Derivation of Categorical Cross-Entropy Loss: A Step-by-Step Probabilistic Foundation

## 1. Probability Foundations

A random variable is a function that maps outcomes of a random experiment to real numbers. In classification, the output of interest is a **discrete random variable** $Y$ that can take one of $C$ possible class labels (for example, $C = 10$ for digit recognition in MNIST).

A **probability distribution** over these classes assigns a probability $p(y = c)$ to each class( $c$, satisfying two conditions:

$$0 \leq p(y = c) \leq 1 \; and \; \sum_{c=1}^{c} p(y = c) = 1$$

When we have a dataset of $N$ independent samples, the joint probability of observing the entire dataset is the product of the individual probabilities. This quantity is called the **likelihood** of the data given the model parameters $\theta$ :

$$L(\theta) \; = \; \prod_{i=1}^{N} p\left( y^{(i)} \mid \; x^{(i)}; \theta \right)$$

Because probabilities are numbers between 0 and 1, their product becomes extremely small for large $N$. To turn the product into a sum (which is easier to optimize) and to preserve monotonicity, we take the **natural logarithm**. The log-likelihood is therefore

$$\log L(\theta) = \; \sum_{i=1}^{N} \log p\left(y^{(i)} \left| x^{(i)} \, ; \theta\right), \right|$$

where $\theta$ denotes the model parameters. Maximizing the log-likelihood is mathematically equivalent to maximizing the likelihood itself, but far more convenient numerically.

## 2. Classification as a Probabilistic Problem

In modern neural networks, the final layer does not output raw class scores; it outputs a **probability distribution** over the $C$ classes. This interpretation is crucial: the network is estimating the conditional probability $p(y = c \, |x; \; \theta)|$ for each input $x$ .

For the outputs to be valid probabilities, they must be non-negative and sum to one. Raw logits $z = [z_1, z_2, \dots, z_C]$ (the linear outputs of the last layer) do not satisfy these constraints. Therefore, we need a normalization function that converts arbitrary real-valued scores into a proper probability distribution.

## 3. Softmax Function

The **softmax** function achieves exactly this transformation. For a vector of logits $z \in R^C$, the softmax probability for class $i$ is

$$\hat{y}_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

The exponential ensures all outputs are strictly positive. The denominator normalizes the values so that $\sum_{i=1}^{c} \hat{y}_i = 1$.

After softmax, the network's output $\hat{y}$ can be interpreted directly as the parameters of a **categorical distribution** over the classes.

## 4. One-Hot Encoding and Ground Truth Representation

The true label for each sample is a single class index $c$ . To express it in the same vector space as the model output, we use **one-hot encoding**:

$$y^{(i)} = [0,0, \dots ,1, \dots ,0]^T$$

where the 1 appears only at the position corresponding to the true class and all other entries are 0. Thus, $y_c^{(i)} \in \{ 0 , 1 \}$ and $\sum_{c=1}^{c} y_c^{(i)} = 1$.

This representation aligns the ground truth with the predicted probability vector, enabling direct comparison between the two distributions.

## 5. Maximum Likelihood Estimation (The Core Reason)

We now write the likelihood of the entire dataset under the assumption that each sample $(x^{(i)}, y^{(i)})$ is drawn independently from a categorical distribution parameterized by the model's softmax outputs.

For a single sample, the probability of observing the true label $y^{(i)} = c$is simply the softmax probability at the correct class. Using the one-hot vector $y^{(i)}$, this is compactly written as

$$p\left(y^{(i)} \middle| x^{(i)}; \theta\right) = \prod_{c=1}^{c} \left(\widehat{y^{(i)}}\right)^{y_c(i)}$$

Because $y^{(i)}$ is one-hot, only one term in the product is raised to the power 1; all others are raised to 0 and thus equal 1, so this expression reduces to $\widehat{y_{c(i)}^{(i)}}$

The likelihood of the full dataset is the product over all $N$ samples:

$L(\theta) = \prod_\{i = 1\}^\{N\}p(y(i) \mid x(i); \theta) = \prod_\{i = 1\}^N\prod_\{c = 1\}^C(\hat\{y\}_c^\{ \vdash(i\dashv )\})^\{^\{_\{y_c\}^\{ \vdash(i\dashv )\}\}\}.$

Taking the logarithm yields the log-likelihood:

$$\log L(\theta) = \sum_{i=1}^{N} \log\left(\prod_{c=1}^{c} \left(\widehat{y_c^{(i)}}\right)^{y_c(i)}\right) = \sum_{i=1}^{N}\sum_{c=1}^{c} y_c^{(i)} \log\left(\widehat{y_c^{(i)}}\right)$$

This is the exact expression that will become the core of our loss function.

## 6. From Log-Likelihood to Loss Function

Optimization libraries are designed to *minimize* objective functions. Because we want to *maximize* the log-likelihood, we simply multiply by −1 to obtain the **negative log-likelihood (NLL)**:

$$L(\theta) = -\log L(\theta) = -\sum_{i=1}^{N}\sum_{c=1}^{C} y_c^{(i)} \log\left(\widehat{y_c^{(i)}}\right).$$

This is precisely the **categorical cross-entropy loss** used in practice. It is **not** an arbitrary formula; it is the direct negative log-likelihood of the observed data under the categorical distribution produced by softmax.

By minimizing *L(θ)*, we are maximizing the probability that the model assigns to the observed labels, which is the standard MLE objective for multi-class classification.

## 7. Cross-Entropy Interpretation

The expression above is also the definition of **cross-entropy** between the true distribution $y$ (one-hot) and the predicted distribution $\hat{y}$:

$$H(y, \hat{y}) = -\sum_{c=1}^{C} y_c \log(\hat{y}_c).$$

ross-entropy measures the **average number of bits** required to encode events from the true distribution using a coding scheme based on the predicted probabilities. In information-theoretic terms, it quantifies how much extra information (or "surprise") is required because the model's probabilities differ from reality.

Since the labels are one-hot and fixed, the **empirical distribution** $y$ is known and does not depend on the model parameters. Thus, minimizing cross-entropy with respect to $\theta$ is equivalent to making the model's predicted distribution $\hat{y}$ as close as possible to the true label distribution

## 8. Why Double Summation Appears

The outer sum in

$$L(\theta) = -\sum_{i=1}^{N}\sum_{c=1}^{C} y_c^{(i)} \log\left(\widehat{y_c^{(i)}}\right)$$

runs over the $N$ training samples. The inner sum runs over the $C$ classes. When the label $y^{(i)}$ is one-hot encoded, only one term in the inner sum is non-zero (the term where $y_c^{(i)} = 1$). All other terms vanish because $y_c^{(i)} = 0$Therefore the loss for a single example simplifies dramatically to

$$L^{(i)} = -\log\left(\widehat{y_{c^{(i)}}^{(i)}}\right)$$

where $c^{(i)}$ is the true class.

The double summation is retained in most derivations and code implementations for **two main reasons**:

- **Vectorization**: It is convenient to write loss functions in terms of full probability vectors, even when the labels are one-hot
- **Generality**: The same expression works if labels are **soft probabilities** (e.g., from label smoothing or multi-label mixtures), not just hard one-hot vectors.

## 9. Why Softmax + Cross-Entropy Work Together

The combination of **softmax** at the output layer and **categorical cross-entropy** as the loss yields an exceptionally simple gradient with respect to the logits $z$.

Let $L$ be the categorical cross-entropy loss for a single example, so $L = -\log\left(\widehat{y_{c^{(i)}}^{(i)}}\right)$The partial derivative of $L$ with respect to a logit $z_k$ is

$$\frac{\partial L}{\partial z_k} = \widehat{y_k} - y_k.$$

That is, the gradient is simply **predicted probability minus true probability** for each class.

This result has several desirable properties:

- **Numerical stability**: The gradient is compact and well-behaved, without large exponents or logarithms.
- **Computational efficiency**: No expensive operations are needed during backpropagation.
- **Built-in stability tricks**: In practice, the softmax is often computed with logit-shifting (e.g., subtracting $max(z)$max(z)) to avoid overflow, and this can be combined with log-sum-exp tricks while preserving the same gradient.

## 10. Common Misconceptions

- The cross-entropy loss is **not** an arbitrary choice; it is the statistically optimal objective under the maximum-likelihood principle for categorical data.
- Mean squared error (MSE) is unsuitable for classification because it assumes Gaussian noise and does not push probabilities toward 0 or 1 aggressively enough. A confident wrong prediction (e.g., $\widehat{y_{correct}} = 0.01$) produces only a modest MSE penalty but an enormous cross-entropy penalty, which is exactly the behavior we want.
- **Misconception 3: "The loss 'just appears in the library'"**

  Every term in the cross-entropy formula traces back to the **likelihood** of the observed labels under the model's probabilistic output. The derivatives are not arbitrary; they follow directly from the chain rule and the softmax–logit relationship

By understanding the derivation outlined above, one sees that categorical cross-entropy is not a magic trick, but the **principled loss** one obtains when treating classification as a probabilistic estimation problem.

## 11. Optional Advanced Section: Relation to Kullback–Leibler Divergence

Cross-entropy can be rewritten using Shannon entropy $H(y)$ and the Kullback–Leibler (KL) divergence $D_{kl}(y \parallel \hat{y})$:

$$H(y, \hat{y}) = H(y) + D_{kl}(y \parallel \hat{y})$$

Here, *H(y)* is the entropy of the true distribution, and $D_{kl}(y \parallel \hat{y})$ measures the extra "surprise" needed when using $\hat{y}$ instead of $y$

Since $y$ is fixed (it is determined by the dataset), *H(y)* is constant with respect to the model parameters $\theta$. Therefore, **minimizing cross-entropy** is *exactly equivalent* to **minimizing the KL divergence** between the true label distribution and the predicted distribution:

$$\min_{\theta} H(y\,,\,\hat{y}) \quad \leftrightarrow \quad \min_{\theta} D_{kl}\,(y \parallel \hat{y})$$

In other words, training with categorical cross-entropy is equivalent to making the model's output distribution as close as possible to the empirical data distribution in the KL sense.

## Conclusion

This derivation shows that the loss

$$Loss \;=\; -\sum_{i=1}^{N}\sum_{c=1}^{c} y_c^{(i)} \log\left(\widehat{y_c^{(i)}}\right)$$

is not merely a convenient formula; it is the **principled negative log-likelihood** of a categorical distribution, equivalent to **cross-entropy**, and directly linked to minimizing **distributional divergence** (via KL).

Understanding this probabilistic and information-theoretic foundation removes the "black-box" feel of categorical cross-entropy and equips practitioners to debug, modify, or extend the loss when needed—for example in label-smoothing, uncertainty-aware training, or custom output distributions

***


1. https://www.geeksforgeeks.org/deep-learning/categorical-cross-entropy-in-multi-class-classification/
2. https://www.machinelearningmastery.com/cross-entropy-for-machine-learning/
3. https://365datascience.com/tutorials/machine-learning-tutorials/cross-entropy-loss/
4. https://ramsane.github.io/articles/cross-entropy-explained-with-entropy-and-kl-divergence/
5. https://www.lightly.ai/blog/cross-entropy-loss
6. https://www.v7labs.com/blog/cross-entropy-loss-guide

7. https://shivammehta25.github.io/posts/deriving-categorical-cross-entropy-and-softmax/

8. https://www.youtube.com/watch?v=dQZgQO1jO88

9. https://www.datacamp.com/tutorial/the-cross-entropy-loss-function-in-machine-learning

10. https://github.com/tensorflow/tensorflow/issues/29081